Deep Convolutional Models: Case Studies

Motivation: Why Study Existing Architectures?

Learning to build effective Convolutional Neural Networks (ConvNets) requires more than understanding the basic building blocks (convolution, pooling, fully connected layers). It requires understanding how to compose them effectively.

- **Building Intuition:** Just as reading open-source code helps a programmer learn to code, analyzing successful neural network architectures helps build intuition for designing your own networks.
- **Transferability:** An architecture that performs well on one computer vision task (e.g., recognizing cats and dogs) often generalizes well to other tasks (e.g., self-driving cars).
    - *Practical Tip:* Instead of designing a network from scratch, it is often effective to take an existing, proven architecture and apply it to a new problem.
- **Research Literacy:** Studying these cases prepares you to read and understand complex computer vision research papers.

Outline of Case Studies

This module will cover several seminal architectures in the history of Computer Vision:

1. Classic Networks

- **LeNet-5:** A pioneering network from the 1980s.
- **AlexNet:** A pivotal network often cited in modern deep learning.
- **VGG:** Known for its simplicity and depth.

2. Advanced Networks

- **ResNet (Residual Network):**
    - Addresses the challenge of training **very deep** networks.
    - Example: Training a 152-layer neural network using specific "tricks" or residual connections.

- **Inception Network:** A complex but highly efficient architecture.

Cross-Disciplinary Impact

The architectural innovations found in networks like **ResNet** and **Inception** are not limited to computer vision. These ideas are increasingly being cross-fertilized into other disciplines, making them valuable concepts even for those not strictly working on vision applications.

Classic Neural Networks

This lecture reviews three influential neural network architectures that established foundational patterns in Computer Vision.

1. LeNet-5 (1998)

Goal: Recognize handwritten digits (0-9) on grayscale images.
Authors: LeCun et al.

Architecture Overview

- **Input:** $32 \times 32 \times 1$ (Grayscale image).
- **Parameters:** ~60,000.
- **Key Components:** Used **Average Pooling** and classic convolutions without padding ("valid" convolutions).

Layer Sequence

1. **Conv 1:** 6 filters (5×5), stride 1. Output: $28 \times 28 \times 6$.
2. **Pool 1:** Average pooling ($f = 2, s = 2$). Output: $14 \times 14 \times 6$.
3. **Conv 2:** 16 filters (5×5), stride 1. Output: $10 \times 10 \times 16$.
4. **Pool 2:** Average pooling ($f = 2, s = 2$). Output: $5 \times 5 \times 16$.
5. **Fully Connected (FC):** Flatten to 400 nodes $\rightarrow$ 120 nodes $\rightarrow$ 84 nodes.
6. **Output:** 10 nodes (Softmax, though original paper used a different classifier).

Historical Notes

- **Activation Functions:** Used **Sigmoid** and **Tanh**. ReLU was not yet standard.
- **Pattern Established:** As you go deeper, height/width (n_H, n_W) decrease, while the number of channels (n_C) increases.
- **Computational Constraints:** The original implementation had complex wiring where distinct filters looked at different input channels to save computation; modern implementations avoid this complexity.

---

2. AlexNet (2012)

Goal: Image classification on the ImageNet dataset (1000 classes).
Authors: Alex Krizhevsky, Ilya Sutskever, Geoffrey Hinton.
Significance: Convinced the Computer Vision community that Deep Learning was viable and effective.

Architecture Overview

- **Input:** $227 \times 227 \times 3$ RGB images (Paper cites 224×224, but math aligns better with 227).
- **Parameters:** ~60 million.
- **Similarities to LeNet:** Structurally similar (Conv $\rightarrow$ Pool $\rightarrow$ FC) but significantly larger and deeper.

Layer Highlights

1. **Conv 1:** 96 filters (11×11), stride 4. Large stride reduces dimensions immediately to 55×55.
2. **Max Pooling:** Uses 3×3 filters with stride 2.
3. **Padding:** Uses "Same" padding in later layers to maintain dimensions.
4. **Output:** Softmax over 1000 classes.

Key Improvements

- **ReLU Activation:** Used ReLU instead of Sigmoid/Tanh, improving training speed and performance.
- **GPUs:** Trained on dual GPUs due to hardware limitations at the time.
- **Local Response Normalization (LRN):** A normalization layer used in the original paper.
    - *Concept:* Normalize activations across channels at a specific (x, y) position to prevent saturation.

- *Status:* Found to be ineffective in later research; rarely used today.

---

3. VGG-16 (2014)

Authors: Simonyan & Zisserman.
Philosophy: Focus on **Simplicity** and **Uniformity**.

Architecture Overview

- **Input:** $224 \times 224 \times 3$.
- **Parameters:** ~138 million (Very large network).
- **Nomenclature:** "16" refers to 16 layers with learnable weights.

Design Rules

Instead of tuning hyperparameters (filter size, stride) for every layer, VGG used a fixed standard:

1. **Convolutions:** Always 3×3 filters, stride 1, **Same** padding.
2. **Max Pooling:** Always 2×2 filters, stride 2.

Layer Sequence Pattern

- **Filter Doubling:** The number of filters doubles systematically after each pooling block: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$.
- **Dimension Halving:** Pooling layers halve the height and width systematically: $224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$.

Pros and Cons

- **Pros:** Extremely uniform and easy to understand architecture.
- **Cons:** Computationally expensive and requires a massive number of parameters (~138M).

Suggested Readings

If interested in the primary literature, the lecturer recommends reading in this order:

1. **AlexNet** (Easier to read).
2. **VGG-16** (Simple principles).
3. **LeNet-5** (Harder to read due to outdated terminology/methods).

Residual Networks (ResNets)

The Problem with Very Deep Networks

While deep networks theoretically should perform better as layers are added, in practice, they are difficult to train due to **vanishing and exploding gradients**.

- **Plain Networks:** As you increase the depth of a standard neural network (a "plain" network), the training error typically decreases for a while but then begins to **increase**.
- **The Solution:** ResNets utilize **Skip Connections** (or Shortcut Connections) to solve this optimization problem, enabling the training of networks with over 100 layers (and sometimes 1000+).

The Residual Block

The fundamental building block of a ResNet is the **Residual Block**.

The "Main Path"

Consider two layers of a neural network where information flows from $a^{[l]}$ to $a^{[l+2]}$:

1. **Step 1:** Linear Operator ($z^{[l+1]} = W^{[l+1]}a^{[l]} + b^{[l+1]}$) $\rightarrow$ ReLU Activation ($a^{[l+1]} = g(z^{[l+1]})$.
2. **Step 2:** Linear Operator ($z^{[l+2]} = W^{[l+2]}a^{[l+1]} + b^{[l+2]}$) $\rightarrow$ ReLU Activation ($a^{[l+2]} = g(z^{[l+2]})$.

The "Short Cut" (Skip Connection)

In a residual block, we copy the activation $a^{[l]}$ and feed it further into the network, injecting it **before** the second ReLU non-linearity.

- **New Equation:**
  $a^{[l+2]} = g(z^{[l+2]} + \mathbf{a}^{[\mathbf{l}]})$
  (Where g is the ReLU activation function)
- **Mechanism:** The value $a^{[l]}$ skips over the intermediate layers and is added directly to $z^{[l+2]}$.
- **Terminology:** The path skipping the layers is called the "short cut" or "skip connection."

Why ResNets Work

- **Identity Mapping:** By adding $a^{[l]}$, it becomes very easy for the block to learn an **identity function** ($a^{[l+2]} = a^{[l]}$). If the additional layers are not useful, the network can easily "ignore" them by setting weights to zero, passing the original information through.
- **Gradient Flow:** The skip connections allow gradients to backpropagate more easily through deep networks, mitigating the vanishing gradient problem.

Building a ResNet

- A full Residual Network is constructed by stacking many of these Residual Blocks together.
- **Performance:** Unlike plain networks, ResNets allow the training error to continue decreasing as the network depth increases.

Why ResNets Work

The Identity Function

The primary reason ResNets function effectively is that their architecture makes it very easy for the network to learn the **identity function**. This ensures that adding extra layers does not hurt performance, and often improves it.

Mathematical Explanation

Consider adding a residual block to a network:

- **Goal:** We want to see if adding two extra layers (a residual block) hurts the network's ability to maintain the performance of a shallower network.
- **Equation:**
  $a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$
  $a^{[l+2]} = g(W^{[l+2]}a^{[l+1]} + b^{[l+2]} + a^{[l]})$
- **Scenario:**
  - Assume we apply regularization (Weight Decay), which shrinks weights $W^{[l+2]}$ towards zero.
  - If $W^{[l+2]} \approx 0$ and $b^{[l+2]} \approx 0$:
    $a^{[l+2]} = g(a^{[l]})$

- Since we use **ReLU** activation (g), and activations $a^{[l]}$ are non-negative: $g(a^{[l]}) = a^{[l]}$
- **Result:** $a^{[l+2]} = a^{[l]}$

Implication

- **Deep Plain Networks:** In standard deep networks, it is difficult for parameters to essentially "zero out" layers to preserve the identity function. This often leads to performance degradation as layers are added.
- **ResNets:** The skip connection makes learning the identity function trivial ($W = 0, b = 0$).
  - *Worst Case:* The extra layers just copy the input, so performance doesn't degrade.
  - *Best Case:* The extra layers learn useful features, improving performance.

Handling Dimensions

The addition $z^{[l+2]} + a^{[l]}$ requires both terms to have the same dimensions.

Case 1: Same Dimensions

- ResNets heavily utilize **Same Convolutions** so that the dimensions of $a^{[l]}$ and $z^{[l+2]}$ match perfectly.
- This allows for direct element-wise addition.

Case 2: Different Dimensions

- If dimensions differ (e.g., due to pooling or changing channel counts), we introduce a matrix W_s.
  $a^{[l+2]} = g(z^{[l+2]} + W_s a^{[l]})$
- **Options for W_s:**
  1. **Learned Parameters:** A matrix that learns to map dimensions (e.g., $128 \to 256$).
  2. **Zero Padding:** A fixed matrix that simply pads the input with zeros to match dimensions.

ResNets on Images

- **Structure:** A typical ResNet consists of many 3×3 **Same Convolutions**.
- **Skip Connections:** Added frequently (every 2 layers).
- **Pooling/Dimension Changes:** Occasionally, pooling layers reduce height/width

(n_H, n_W). When this happens, the matrix W_s is used to adjust dimensions for the skip connection.

- **Output:** Ends with a Fully Connected layer and Softmax.

Networks in Networks and 1x1 Convolutions

The Concept of 1x1 Convolutions

At first glance, a 1×1 convolution might seem trivial—simply multiplying an image by a number. While this is true for a single-channel image (e.g., $6 \times 6 \times 1$), the operation becomes powerful when applied to **multi-channel volumes**.

Mechanism

Consider an input volume of dimension $6 \times 6 \times 32$.

- **The Filter:** A 1×1 filter technically has dimensions $1 \times 1 \times 32$ (matching the input channels).
- **The Operation:**
  1. The filter iterates over every spatial position (all 36 pixels).
  2. At each position, it takes the 32 numbers (from the channels), performs an element-wise multiplication with the filter weights, and sums them.
  3. A **ReLU** non-linearity is applied.
  4. **Result:** A single real number for that pixel position.
- **Output:** If you use multiple filters (e.g., equal to the number of desired output channels), the output volume will be $6 \times 6 \times \#\text{filters}$.

Interpretation: "Network in Network"

- You can view a 1×1 convolution as a **Fully Connected Neural Network** applied to each pixel position independently.
- It takes the n_C input channels at a specific location, multiplies them by weights (the filter), and outputs a new value.
- This concept is derived from the "Network in Network" paper (Lin, Chen, & Yan), which has significantly influenced modern architectures like **Inception**.

Utility and Applications

Why use a 1×1 convolution if it doesn't change the height or width?

1. Dimension Reduction (Shrinking Channels)

- **Pooling Layers:** Used to shrink height (n_H) and width (n_W).
- **1x1 Convolutions:** Used to shrink the number of channels (n_C).
- **Example:**
  - **Input:** $28 \times 28 \times 192$ (High computational cost for subsequent layers).
  - **Operation:** Convolve with **32** filters of size $1 \times 1 \times 192$.
  - **Output:** $28 \times 28 \times 32$.
- **Benefit:** This reduces the volume size and computational cost while retaining the spatial resolution.

2. Increasing Non-Linearity

- Even if you do not want to shrink dimensions (e.g., mapping 192 channels to 192 channels), adding a 1×1 convolution layer adds a learnable layer with **ReLU activation**.
- This allows the network to learn more complex functions without changing the spatial dimensions of the feature map.

Inception Network Motivation

The Core Concept

When designing a ConvNet layer, selecting the hyperparameters can be difficult. Should you use a 1×1, 3×3, or 5×5 filter? Or should you use a pooling layer?

The **Inception Network** (GoogleLeNet) proposes a solution: **Do them all.**

The Naive Inception Module

Instead of choosing a single filter size, the Inception module applies multiple operations in parallel to the *same* input volume and stacks the results.

- **Operations:**
  1. 1×1 Convolution
  2. 3×3 Convolution

3. 5×5 Convolution
4. Max Pooling

- **Concatenation:** The outputs of these four operations are concatenated along the channel dimension to form a single output volume.
- **Constraint:** To stack the volumes, their height and width (n_H, n_W) must match.
  - **Convolutions:** Use **"Same"** padding.
  - **Max Pooling:** Use specific padding and stride ($s = 1$) to maintain input dimensions (this is unusual for pooling but necessary here).

The Problem: Computational Cost

While the naive approach allows the network to learn which features are most useful, it is extremely computationally expensive.

Example Calculation:
Consider a 5×5 convolution within the module:

- **Input:** $28 \times 28 \times 192$
- **Filter:** 32 filters of size $5 \times 5 \times 192$
- **Output:** $28 \times 28 \times 32$

Operations Count:
To calculate the total multiplications, multiply the number of output values by the number of operations per value:
$(\text{Output Size}) \times (\text{Filter Size}) \times (\text{Input Channels})$
$(28 \times 28 \times 32) \times (5 \times 5 \times 192) \approx \mathbf{120}$ million

Result: 120 million multiplications for just *one* component of *one* layer is prohibitively expensive.

The Solution: Bottleneck Layers

To solve the computational cost problem without losing the benefits of the architecture, Inception networks use 1×1 **convolutions** to reduce the number of channels *before* applying expensive convolutions.

The Bottleneck Mechanism

This architecture inserts a 1×1 convolution before the 5×5 convolution to shrink the channel depth.

- **Analogy:** A bottleneck is the narrowest part of a bottle. Similarly, this layer

forces the data through a smaller representation (fewer channels) before expanding it again.

Optimized Calculation (With Bottleneck)

Let's recalculate the cost using a bottleneck to reduce channels from 192 to 16 before the 5×5 conv.

Step 1: 1×1 Convolution (The Bottleneck)

- **Input:** $28 \times 28 \times 192$
- **Filter:** 16 filters of size $1 \times 1 \times 192$
- **Output:** $28 \times 28 \times 16$
- **Cost:** $(28 \times 28 \times 16) \times (1 \times 1 \times 192) \approx \mathbf{2.4}$ million

Step 2: 5×5 Convolution

- **Input:** $28 \times 28 \times 16$ (The output from Step 1)
- **Filter:** 32 filters of size $5 \times 5 \times 16$
- **Output:** $28 \times 28 \times 32$ (Same final dimensions as the naive version)
- **Cost:** $(28 \times 28 \times 32) \times (5 \times 5 \times 16) \approx \mathbf{10.0}$ million

Total Cost:
$2.4 \text{ M} + 10.0 \text{ M} = \mathbf{12.4}$ million

Comparison

- **Naive Approach:** 120 Million operations.
- **Bottleneck Approach:** 12.4 Million operations.
- **Impact:** The computational cost is reduced to roughly **1/10th** of the original without significantly hurting performance.

The Inception Network

Building the Inception Module

The Inception Network is built by stacking **Inception Modules**. A single module runs multiple operations in parallel on the same input volume and concatenates the results.

Component Breakdown

Consider an input volume of $28 \times 28 \times 192$. The module applies the following parallel branches:

1. **1x1 Conv Branch:**
    - Applies 1×1 filters (e.g., 64 filters).
    - Output: $28 \times 28 \times 64$.
2. **3x3 Conv Branch:**
    - First, applies a **1x1 Conv** (bottleneck) to reduce channels (e.g., to 96).
    - Then, applies **3x3 Conv** (e.g., 128 filters).
    - Output: $28 \times 28 \times 128$.
3. **5x5 Conv Branch:**
    - First, applies a **1x1 Conv** (bottleneck) to reduce channels (e.g., to 16).
    - Then, applies **5x5 Conv** (e.g., 32 filters).
    - Output: $28 \times 28 \times 32$.
4. **Max Pooling Branch:**
    - Applies **Max Pooling** ($f = 3$, $s = 1$, padding="same").
        - *Note:* Standard pooling preserves channel depth, so the output would be $28 \times 28 \times 192$. This is too large.
    - **Crucial Step:** Apply a **1x1 Conv** *after* pooling to shrink the channel count (e.g., to 32).
    - Output: $28 \times 28 \times 32$.

Channel Concatenation

The outputs of all four branches are concatenated along the channel dimension.

- **Total Channels:** $64 + 128 + 32 + 32 = 256$.
- **Final Output Volume:** $28 \times 28 \times 256$.

The GoogleLeNet Architecture

The full Inception Network (often called **GoogleLeNet**) consists of these inception modules connected in series.

- **Structure:**
    - Standard Convolution and Pooling layers at the very beginning (stem).
    - Stacked **Inception Modules** repeatedly throughout the network.

- Occasional Max Pooling layers between modules to reduce height and width (n_H, n_W).
- Ends with a Fully Connected layer and Softmax.

Side Branches (Auxiliary Classifiers)

A unique feature of the original Inception network is the presence of **Side Branches**.

- **Location:** Attached to hidden layers in the middle of the network.
- **Components:** They consist of their own small sub-networks (5x5 pooling, 1x1 conv, FC layers) ending in a Softmax classifier.
- **Purpose:**
  1. **Regularization:** They help prevent overfitting.
  2. **Feature Quality:** They ensure that the features learned at intermediate layers are discriminative enough to predict the output class, effectively injecting gradients earlier in the network.

Historical Context & Variations

- **Naming:** The name "Inception" is derived from the "We need to go deeper" meme from the movie *Inception*, which the authors cited in the paper.
- **Variations:**
  - **Inception v2, v3, v4:** Improved versions with various optimizations.
  - **Inception-ResNet:** A hybrid architecture combining Inception modules with the Skip Connections found in ResNets.

MobileNets and Depthwise Separable Convolutions

Motivation

Standard Convolutional Neural Networks (like ResNet or Inception) are computationally expensive. **MobileNets** are designed for low-compute environments, such as mobile phones or embedded devices, offering a lighter-weight alternative with comparable performance for many tasks.

The core innovation that enables this efficiency is the **Depthwise Separable Convolution**.

1. The Baseline: Standard Convolution

To understand the savings, first consider the cost of a standard convolution.

- **Input:** $n \times n \times n_c$ (e.g., $6 \times 6 \times 3$)
- **Filter:** $f \times f \times n_c$ (e.g., $3 \times 3 \times 3$)
- **Number of Filters:** n_c' (e.g., 5)
- **Output:** $n_{out} \times n_{out} \times n_c'$ (e.g., $4 \times 4 \times 5$)

Computational Cost Formula

The cost is determined by the number of multiplications required:
$\text{Cost} = f \cdot f \cdot n_c \cdot n_{out} \cdot n_{out} \cdot n_c'$

- **Example Calculation:**
  $3 \cdot 3 \cdot 3 \cdot 4 \cdot 4 \cdot 5 = \mathbf{2,160}$ multiplications

---

2. Depthwise Separable Convolution

This operation replaces the standard convolution with two distinct steps: **Depthwise Convolution** followed by **Pointwise Convolution**.

Step 1: Depthwise Convolution

Instead of combining all channels simultaneously, we apply a single filter to each input channel independently.

- **Input:** $n \times n \times n_c$ ($6 \times 6 \times 3$)
- **Filters:** $f \times f \times 1$. We have n_c filters (one for each channel).
- **Operation:** Convolve the red filter with the red channel, green with green, etc.
- **Output:** $n_{out} \times n_{out} \times n_c$ ($4 \times 4 \times 3$). Note that the number of channels does not change yet.
- **Step 1 Cost:**
  $f \cdot f \cdot n_{out} \cdot n_{out} \cdot n_c$
  - *Example:* $3 \cdot 3 \cdot 4 \cdot 4 \cdot 3 = \mathbf{432}$

Step 2: Pointwise Convolution

To combine the features across channels (which the depthwise step did not do), we use a 1×1 convolution.

- **Input:** $n_{out} \times n_{out} \times n_c$ (Output from Step 1)
- **Filters:** $1 \times 1 \times n_c$. We have n_c' filters (to match the desired output depth).

- **Operation:** Standard 1×1 convolution across all channels.
- **Output:** $n_{out} \times n_{out} \times n'_c$ $(4 \times 4 \times 5)$.
- **Step 2 Cost:**
  $1 \cdot 1 \cdot n_c \cdot n_{out} \cdot n_{out} \cdot n'_c$
  - *Example:* $1 \cdot 1 \cdot 3 \cdot 4 \cdot 4 \cdot 5 = \mathbf{240}$

---

3. Computational Cost Comparison

Total Cost of Depthwise Separable Convolution:
$\text{Cost}_{\text{depthwise}} + \text{Cost}_{\text{pointwise}} = 432 + 240 = \mathbf{672}$

Comparison:

- **Standard Conv:** 2,160 ops
- **Depthwise Separable:** 672 ops
- **Ratio:** $\frac{672}{2160} \approx 0.31$ (The new method is roughly 31% of the cost).

General Savings Formula

The reduction in computational cost can be expressed as:
$\frac{\text{Cost}_{\text{separable}}}{\text{Cost}_{\text{standard}}} = \frac{1}{n'_c} + \frac{1}{f^2}$

- In typical networks where n'_c (output channels) is large, the term $\frac{1}{n'_c}$ becomes negligible.
- The cost is dominated by $\frac{1}{f^2}$.
- **Implication:** For a 3×3 filter, the depthwise separable convolution is roughly **8 to 9 times cheaper** than a standard convolution.

Notation Note

For future diagrams in this course:

- The icon for Depthwise Convolution will be simplified to a stack of filters (like a $3 \times 3 \times 3$ stack), even if n_c is much larger.
- The icon for Pointwise Convolution will be shown as a 1×1 filter stack (pink block).

MobileNet Architecture

MobileNet v1

MobileNet v1 is designed to replace computationally expensive standard convolutions with **Depthwise Separable Convolutions** throughout the network.

- **Core Block:** A single block consists of a **Depthwise Convolution** followed by a **Pointwise Convolution**.
- **Architecture:** The network stacks this core block **13 times**.
- **Final Layers:** After the convolutional blocks, the network uses:
    1. Average Pooling
    2. Fully Connected (FC) Layer
    3. Softmax (for classification)

MobileNet v2: Key Enhancements

MobileNet v2 (Sandler et al.) introduced two significant modifications to the v1 architecture to improve performance while maintaining low resource usage.

1. Residual Connections

Like ResNets, MobileNet v2 uses **skip connections** (residual connections). This allows gradients to propagate backward more efficiently during training by summing the input of a block with its output.

2. The Bottleneck Block (Expansion + Projection)

The most significant change in v2 is the introduction of the **Bottleneck Block**. This block consists of three distinct steps:

1. **Expansion Layer:** A 1×1 convolution that increases the number of channels (typically by an **expansion factor of 6**).
    - *Purpose:* Increases the representation size to allow the network to learn richer, more complex functions.
    - *Example:* An input of $n \times n \times 3$ is expanded to $n \times n \times 18$.
2. **Depthwise Convolution:** A depthwise separable convolution is applied to the expanded volume. Padding is used to ensure the spatial dimensions ($n \times n$) do not shrink.
3. **Projection Layer (Pointwise Convolution):** A 1×1 convolution that "projects"

the high-dimensional representation back down to a smaller number of channels.
    - *Purpose:* Reduces the memory footprint required to store activations for the next block.

Architecture Summary Comparison

Feature	MobileNet v1	MobileNet v2
Primary Block	Depthwise Separable Conv	Bottleneck Block (Expansion/Proj)
Main Stacking	13 Blocks	17 Blocks
Skip Connections	No	Yes (Residual Connections)
Memory Efficiency	High	Higher (due to projection)
Complexity Capacity	Moderate	High (due to expansion)

Motivation for the Bottleneck

The "clever" aspect of the bottleneck block is the balance between computation and memory:

- **Expansion** enables a **richer set of computations** internally.
- **Projection** keeps the **activation size small** when passing data between layers, which is critical for mobile devices with heavy memory constraints.

EfficientNet: Scaling Neural Networks

The Problem: Manual vs. Automatic Scaling

While MobileNets provide efficient layers, developers often need to adjust a model to specific hardware (e.g., a high-end flagship phone vs. a low-power edge device).

Usually, to increase accuracy, researchers manually increase one of three dimensions:

1. **Resolution (r):** Using higher-resolution input images (e.g., $224 \times 224 \rightarrow 299 \times 299$).
2. **Depth (d):** Adding more layers to the network (e.g., ResNet-50 $\rightarrow$ ResNet-101).
3. **Width (w):** Increasing the number of channels or hidden units in each layer.

The Challenge: It is unclear what the best ratio is. For example, if you double the depth but keep the resolution small, you might reach a point of diminishing returns.

Compound Scaling

The authors of EfficientNet (Tan and Le) observed that scaling these three dimensions (r, d, w) in a **coordinated way** produces much better results than scaling any single dimension independently.

- **Concept:** If you have a higher computational budget, you should scale up the image resolution, the network depth, and the layer width **simultaneously** using a fixed set of scaling coefficients.
- **The "Compound" Effect:** A higher-resolution image needs a deeper network (to increase the receptive field) and a wider network (to capture more fine-grained patterns).

Implementation in Practice

EfficientNet provides a family of models (B0 through B7) that are already optimized for different computational budgets.

- **B0:** The baseline model (computationally light).
- **B7:** A scaled-up version (higher accuracy, requires more compute).
- **Usage:** If you are deploying to a specific device, you can look at open-source implementations of EfficientNet to find the version that fits your **computational budget** (latency and memory) while maximizing **accuracy**.

Summary of Week 2 Case Studies

You have now covered several major breakthroughs in CNN architectures:

- **Classic Nets:** LeNet, AlexNet, VGG.
- **Residual Nets:** ResNets for training very deep models.

- **Inception:** Using multiple filter sizes in parallel.
- **MobileNet:** Depthwise separable convolutions for efficiency.
- **EfficientNet:** Compound scaling for device-specific optimization.